Dataset & Project Insights

Real Estate Investment Advisor — Predicting Property Profitability & Future Value

Prepared 30 August 2026 | Source: Datasets/india_housing_prices_with_target_columns.csv, EDA.ipynb, Models.ipynb, README.md

---

This note summarizes what the 250,000-row India housing dataset behind the app actually contains, how its two prediction targets were built, and what that means for interpreting the app's output. It is meant as a companion for walking someone through the project.

1. Project overview

The app takes a property's details — BHK, size, current price, floor, age, amenities, city, and a few categorical factors — and returns two predictions: an **estimated price five years out** (regression) and a **“Good Investment” verdict** (classification), served by tuned LightGBM models. Four EDA pages precede the prediction page, covering distributions, correlations, categorical breakdowns, and investment-factor analysis.

STAGE	ARTIFACT
Data prep & feature engineering	EDA.ipynb
Model training & selection	Models.ipynb
Experiment tracking	MLflow.ipynb, mlflow.db
Serving	Home.py + pages/ (Streamlit)

2. Dataset anatomy

250,000 rows × 26 columns (76 after one-hot encoding), covering **20 states / 42 cities / 500 localities**. No missing values, no exact duplicate rows.

Numeric fields

COLUMN	RANGE	MEAN	SHAPE
Price_in_Lakhs	10 – 500	254.6	Flat / uniform
Size_in_SqFt	500 – 5,000	2,750	Flat / uniform
BHK	1 – 5	3.0	Flat / uniform
Year_Built	1990 – 2023	2006.5	Flat / uniform
Age_of_Property	2 – 35	18.5	= 2025 – Year_Built, exactly
Floor_No / Total_Floors	0–30 / 1–30	15.0 / 15.5	Flat / uniform, sampled independently
Nearby_Schools / Hospitals	1 – 10	5.5	Flat / uniform
Amenities_Count	1 – 5	3.0	Derived from Amenities list length

Every numeric field is close to *perfectly* uniform across its stated range — property size has a skew of 0.0008, price a skew of 0.008. Real housing markets cluster around typical unit sizes and price points; this dataset does not, which is the first sign it was generated rather than observed.

Categorical fields

Twelve columns (State, City, Locality, Property_Type, Furnished_Status, Public_Transport_Accessibility, Parking_Space, Security, Amenities, Facing, Owner_Type, Availability_Status). Every one splits almost exactly evenly across its categories — e.g. Property_Type is 83,744 / 83,300 / 82,956 across Villa / Independent House / Apartment. That level of balance doesn't occur in scraped listings data; it's the fingerprint of stratified synthetic sampling.

3. How the two targets are built

The load-bearing finding of this document: **both prediction targets are deterministic formulas over other columns, not observed market outcomes.** Confirmed by reading `EDA.ipynb`'s data-generation cells and verifying the formulas directly against the data.

Future_Price_5Y (regression target)

```
growth_rate = city_tier_rate(City)          # 0.055 - 0.11, four city
tiers
            + property_type_effect(Type)    # Villa +0.010, House +0.007,
Apartment +0
            - 0.0015 x Age_of_Property
            + Normal(0, 0.018)              # noise, seed=42

Future_Price_5Y = Price_in_Lakhs x (1 + growth_rate) ^ 5
```

Verified directly: for every sampled row, `Price_in_Lakhs × (1 + Growth_Rate_Annual)^5` reproduces `Future_Price_5Y` to 10 decimal places. There is no market signal here — it's compound interest applied to a rate drawn mostly from which city a property is in.

Good_Investment (classification target)

```
score =  1  if Price_per_SqFt <= city median Price_per_SqFt
      +  1  if BHK >= 3
      +  1  if Availability_Status == "Ready_to_Move"
      +  1  if Parking_Space == "Yes" AND Amenities_Count >= 3

Good_Investment = 1  if score >= 3   (else 0)
```

27.6% of properties clear the ≥3-of-4 bar. `Growth_Rate_Annual` — the thing that determines future price — plays **no role** in this label at all.

4. What actually drives each target

Because both targets are formulas, “feature importance” here means recovering the formula, not discovering market behavior.

Future_Price_5Y — correlation with numeric features: Price_in_Lakhs +0.962, Price_per_SqFt +0.534, Good_Investment −0.246, Growth_Rate_Annual +0.243, Year_Built +0.121. Everything else (BHK, size, floor, schools, hospitals, amenities) sits under 0.003. The deployed LightGBM regressor's grouped feature importance confirms this: **City** (3,428 split gain, the growth-tier lookup), **Price_in_Lakhs** (1,125), and **Age_of_Property** (730) dominate, out of 18 grouped features.

Good_Investment — correlation with numeric features: BHK +0.312, Price_per_SqFt −0.256, Price_in_Lakhs −0.256, Size_in_SqFt +0.204, Amenities_Count +0.189.

“Good Investment” rate by the four scoring conditions — each shows a step function right at the rule's threshold, not a gradient, because these *are* the conditions the label was built from:

CONDITION	BELOW THRESHOLD	AT/ABOVE THRESHOLD
Availability status	Under_Construction: 9.2%	Ready_to_Move: 45.9%
Parking space	No: 15.5%	Yes: 39.7%
BHK	1–2 BHK: ~7.8%	3–5 BHK: ~40.8%
Amenities count	1–2: ~15.6%	3–5: ~35.5%

5. Data quality notes

Structurally clean (no nulls, no duplicate rows), but two fields don't hold together internally.

CLEAN

Zero missing values, zero exact duplicate rows across all 250,000 × 26 cells.

FLAG

Floor_No exceeds Total_Floors in 116,304 rows (46.5%). Floor and total-floors are sampled independently rather than floor ≤ total, so nearly half the dataset describes a property on, say, floor 22 of a 5-floor building. Harmless for the current models, but would break any feature that used these fields literally.

FLAG

Price_per_SqFt doesn't equal Price_in_Lakhs ÷ Size_in_SqFt. Spot-checking rows shows the stated Price_per_SqFt (range 0–0.99, mean 0.13) is unrelated to the two fields it should be derived from — e.g. one row has Price ≈ ₹109L over 3,342 sqft (implying ≈ ₹3,233/sqft) but a stated Price_per_SqFt of 0.03. It's an independently sampled field that happens to share a name, not a computed ratio. The Good_Investment rule (§3) uses it as one of its four conditions regardless.

BY DESIGN

Age_of_Property is fully redundant with Year_Built. Year_Built + Age_of_Property = 2025 for all 250,000 rows, std. dev. 0.0. Not a bug — the dataset has a fixed reference year — but the two columns carry identical information and inflate the regression model's Age_of_Property importance with what is really Year_Built's signal split in two.

6. **Modeling results**

LightGBM wins both tasks before any tuning; tuning only widens the lead. Deployed as `real_estate_regression_model_v5` / `real_estate_classification_model_v5`.

Regression — Future_Price_5Y

MODEL	R^2	RMSE (L)	MAE (L)
Linear / Ridge	0.960	39.05	29.32
Random Forest	0.967	35.43	24.71
XGBoost	0.970	33.82	23.47
LightGBM lean, tuned (deployed)	**0.971**	**33.38**	**23.22**

R^2 above 0.96 for *every* model tried, including plain linear regression, is itself informative: it's the signature of a target that's a near-linear transform of one input feature (Price_in_Lakhs) rather than a hard prediction problem.

Classification — Good_Investment

(target imbalanced: 27.6% positive)

MODEL	ROC-AUC	F1	RECALL	ACCURACY
Gradient Boosting	0.722	0.029	0.015	0.723
Logistic Regression	0.723	0.552	0.713	0.681
LightGBM lean, tuned (deployed)	**0.721**	**0.581**	**0.834**	**0.669**

Gradient Boosting's 72.3% accuracy is a trap — 1.5% recall means it almost never predicts "Good Investment." LightGBM trades ~5 points of raw accuracy for 83% recall and the best F1/ROC-AUC of anything tried, which is why it was selected despite not topping the accuracy column.

7. Implications & recommendations

- **Treat this as a demonstration pipeline, not a market tool.** Both targets are synthetic formulas over the dataset's own columns, so the app showcases an ML workflow (EDA → feature engineering → model selection → MLflow tracking → Streamlit serving) rather than real Indian real-estate forecasting.
- **The 5-year price estimate is arithmetic, not a forecast.** Since Future_Price_5Y ≈ Price_in_Lakhs × (1 + f(City, Type, Age))^5, changing only BHK, floor, or amenities on the Prediction page will barely move this number — City, current price, and age are what matter.
- **The "Good Investment" verdict is a rule-recovery exercise.** It's genuinely learnable (F1 0.58, recall 83%) precisely because it's a deterministic 4-condition score. It says "does this listing match a value/availability heuristic," not "will this appreciate" — the two targets are close to independent (corr −0.25).
- **Fix or drop Floor_No / Total_Floors and Price_per_SqFt** before adding new features built on floor ratio or price-per-sqft consistency.
- **Drop Age_of_Property or Year_Built, not both** — they're a perfect duality.
- **For real predictive value**, swap the synthetic Growth_Rate_Annual formula for an actual observed outcome (real transacted price history or a market index); the modeling pipeline would carry over unchanged.